

SQL Avanzado

1º DAM — Bases de datos · Resumen Tema 5

1. Gestión de usuarios

Crear usuario	<code>CREATE USER 'user'@'localhost' IDENTIFIED BY 'pass';</code> Usar % en lugar de localhost permite conexión desde cualquier IP.
Eliminar usuario	<code>DROP USER 'user'@'localhost';</code>
Asignar privilegios	<code>GRANT ALL PRIVILEGES ON bd.* TO 'user'@'localhost';</code> <code>GRANT SELECT, INSERT ON bd.tabla TO 'user'@'localhost';</code> Asignar solo los privilegios necesarios (principio de mínimo privilegio).
Quitar privilegios	<code>REVOKE INSERT ON bd.tabla FROM 'user'@'localhost';</code>
Ver usuarios del sistema	<code>SELECT user, host, db, Select_priv... FROM mysql.user;</code>
Ver privilegios de un usuario	<code>SHOW GRANTS FOR 'user'@'localhost';</code>

2. Transacciones

Concepto	Unidad atómica de trabajo. Si todos los cambios tienen éxito → COMMIT. Si falla alguno → ROLLBACK y se deshace todo.
Inicio	<code>START TRANSACTION;</code> o <code>BEGIN;</code>
Confirmar	<code>COMMIT;</code> → hace los cambios permanentes.
Deshacer	<code>ROLLBACK;</code> → cancela todos los cambios de la transacción.
Autocommit	<code>SET autocommit=0;</code> / <code>SET autocommit=1;</code> Por defecto MySQL tiene autocommit=1: cada instrucción individual es atómica automáticamente.
DDL y transacciones	Las sentencias DDL (CREATE, DROP, ALTER...) NO pueden revertirse y cierran la transacción implícitamente.

3. Procedimientos almacenados y funciones

3.1 Procedimientos almacenados

Concepto	Bloque de sentencias SQL guardado en el servidor. La primera ejecución compila y guarda en caché; las siguientes usan la caché.
----------	---

Crear	<pre> DELIMITER \$\$ CREATE PROCEDURE Nombre() BEGIN -- instrucciones; END\$\$ DELIMITER ; </pre>
Invocar	<pre> CALL Nombre(); </pre>

3.2 Delimitador (DELIMITER)

El cliente MySQL usa ; para separar y enviar instrucciones. Como los procedimientos contienen ; internos, hay que cambiar temporalmente el delimitador (\$\$ o //) para que todo el bloque se envíe como una sola sentencia.

3.3 Variables

Declarar	<pre> DECLARE nombre tipo(tamaño) [DEFAULT valor]; </pre> Sin DEFAULT el valor inicial es NULL. Se pueden declarar varias del mismo tipo juntas.
Asignar con SET	<pre> SET nombre_variable = valor; </pre>
Asignar con SELECT	<pre> SELECT COUNT(*) INTO miVar FROM Tabla; </pre>

3.4 Parámetros

Modo	Descripción
IN (defecto)	El procedimiento recibe el valor pero no puede modificar el original.
OUT	El procedimiento devuelve un valor al llamador.
INOUT	Entrada y salida a la vez.

Nota: en funciones almacenadas todos los parámetros son IN; no se pueden declarar como OUT o INOUT.

3.5 Funciones almacenadas

Concepto	Programa almacenado que devuelve un único valor. Uso con CREATE FUNCTION ... RETURNS tipo.
Estructura	<pre> CREATE FUNCTION nombre(p tipo) RETURNS tipo BEGIN -- instrucciones RETURN valor; END\$\$ </pre> Al llegar a RETURN la ejecución termina inmediatamente.

3.6 Instrucciones condicionales

IF ... THEN ... END IF	<pre> IF condicion THEN instrucciones; END IF; </pre>
------------------------	---

IF ... THEN ... ELSE ... END IF	<pre>IF condicion THEN instrucciones; ELSE instrucciones_else; END IF;</pre>
CASE ... WHEN ... END CASE	<pre>CASE valor WHEN v1 THEN instrucciones WHEN v2 THEN instrucciones [ELSE instrucciones] END CASE;</pre>

Bucles

Bucle	Cuándo comprueba	Mínimo de ejecuciones	Salida
LOOP	Manual (con LEAVE)	0 (si entra con LEAVE inmediato)	LEAVE etiqueta;
WHILE	Al inicio (pre-prueba)	0	Condición falsa
REPEAT	Al final (post-prueba)	1 (siempre)	UNTIL condicion

3.8 Cursores

Permiten recorrer fila a fila el resultado de un SELECT dentro de un procedimiento almacenado.

Paso	Instrucción	Descripción
1 · Declarar variables	DECLARE var tipo;	Primero siempre las variables.
2 · Declarar cursor	DECLARE nombre CURSOR FOR SELECT ...;	Asociado a un SELECT.
3 · Declarar handler	DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin=1;	Marca fin del resultado.
4 · Abrir	OPEN nombre;	Inicializa el resultado.
5 · Leer en bucle	FETCH nombre INTO var;	Recupera la siguiente fila.
6 · Cerrar	CLOSE nombre;	Libera la memoria asociada.

4. Triggers (disparadores)

Concepto	Programa almacenado que se ejecuta automáticamente ante un evento INSERT, UPDATE o DELETE en una tabla. Se activa por cada fila afectada.
Sintaxis	<pre>CREATE TRIGGER nombre {BEFORE AFTER} {INSERT UPDATE DELETE} ON tabla FOR EACH ROW instrucciones;</pre>

Modificadores OLD y NEW

Evento	OLD disponible	NEW disponible
INSERT	NO	SÍ
UPDATE	SÍ	SÍ
DELETE	SÍ	NO

Consideraciones importantes

Recursividad	Si un trigger modifica una tabla que activa otro trigger se puede generar un ciclo infinito. Muchos SGBD permiten desactivarla.
Rendimiento	Pueden afectar negativamente en BBDD con alto volumen de transacciones. Mantener el código simple y limitar operaciones de lectura/escritura.
Debugging	El código dentro de un trigger es más difícil de depurar y mantener que el código en una aplicación (Python, Java...).
Coordinación entre capas	Los triggers ejecutan lógica directamente en la BD. Si el trigger es muy complejo algorítmicamente, esa lógica debería moverse a la capa controlador de la aplicación.

Usos habituales de triggers

Validaciones complejas	Reglas de negocio que van más allá de las restricciones básicas (claves primarias, UNIQUE...). Ej.: salario de un empleado no supera el presupuesto de su departamento.
Auditoría de cambios	Registrar automáticamente quién y cuándo modificó un registro, guardando el valor antiguo en una tabla de historial.
Manejo de errores	Implementar validaciones sólidas para hacer la BD más robusta ante datos incorrectos o acciones no deseadas.
Triggers temporales	Algunos SGBD (ej. SQL Server) permiten triggers que se autoeliminan tras un periodo. Útil para campañas de marketing o auditorías puntuales.